

```
int cmp ( const void *a , const void *b ) {
    return *(int *)a - *(int *)b;
}
```

一般地，需要先把参数 a 和 b 转化为真实的类型，然后让 `cmp` 函数当 $a < b$ 、 $a = b$ 和 $a > b$ 时分别返回负数、0 和正数即可。学会排序之后，本题的主程序并不难编写，读者不妨一试。

是不是觉得上面那个 `cmp` 看起来非常别扭？的确如此。虽然 `qsort` 是 C 语言的标准库函数，但在算法竞赛中一般不使用它，而是使用 C++ 中的 `sort` 函数。此函数将在第 5 章中介绍。本节的主要目的是告诉读者，“将一个函数作为参数传递给另外一个函数”是很有用的。

4.3 递归

终于到了本书 C 语言部分的最后一站——递归了。很多人都认为递归是语言中最难理解的内容之一，但也不要紧张：如果认真理解了 4.2 节中的指针、地址和调用栈，会发现递归其实是一个很自然的东西。

4.3.1 递归定义

递归的定义如下：

递归：

参见“递归”。

什么？这个定义什么也没有说啊！好吧，改一下：

递归：

如果还是没明白递归是什么意思，参见“递归”。

噢，也许这次你明白了，原来递归就是“自己用到自己”的意思。这个定义显然比上一个要好些，因为当你终于悟出其中的道理后，就不必继续“参见”下去了。事实上，递归的含义比这要广泛。

A 经理：“这事不归我管，去找 B 经理。”于是你去找 B 经理。

B 经理：“这事不归我管，去找 A 经理。”于是你又回到了 A 经理这儿。

接下来发生的事情就不难想到了。只要两个经理的说辞不变，你又始终听话，你将会永远往返于两个经理之间。这叫做无限递归（Infinite Recursion）。尽管在这里，A 经理并没有让你找他自己，但还是回到了他这里。换句话说，“间接地用到自己”也算递归。

回忆一下，正整数是如何定义的？正整数是 1, 2, 3, …… 这些数。这样的定义也许对于小学生来说是没有任何问题的，但当你开始觉得这个定义“不太严密”时，你或许会喜欢这样的定义：

(1) 1 是正整数。

(2) 如果 n 是正整数， $n+1$ 也是正整数。

(3) 只有通过(1)、(2)定义出来的才是正整数^①。

这样的定义也是递归的：在“正整数”还没有定义完时，就用到了“正整数”的定义。这和前面的“参见递归”在本质上是相同的，只是没有它那么直接和明显。

同样地，可以递归定义“常量表达式”（以下简称表达式）：

(1) 整数和浮点数都是表达式。

(2) 如果 A 是表达式，则 (A) 是表达式。

(3) 如果 A 和 B 都是表达式，则 A+B、A-B、A*B、A/B 都是表达式。

(4) 只有通过(1)、(2)、(3)定义出来的才是表达式。

简洁而严密，这就是递归定义的优点。

4.3.2 递归函数

数学函数也可以递归定义。例如，阶乘函数 $f(n)=n!$ 可以定义为：

$$\begin{cases} f(0)=1 \\ f(n)=f(n-1)\times n \quad (n\geq 1) \end{cases}$$

对应的程序如下：

程序 4-10 用递归法计算阶乘

```
#include<stdio.h>
int f(int n)
{
    return n == 0 ? 1 : f(n-1)*n;
}
int main()
{
    printf("%d\n", f(3));
    return 0;
}
```

提示 4-17：C 语言支持递归，即函数可以直接或间接地调用自己。但要注意为递归函数编写终止条件，否则将产生无限递归。

4.3.3 C 语言对递归的支持

尽管从概念上可以理解阶乘的递归定义，但在 C 语言中函数为什么真的可以“自己调用自己”呢？下面再次借助 gdb 来调试这段程序。

首先用 b f 命令设置断点——除了可以按行号设置外，也可以直接给出函数名，断点将设置在函数的开头。下面用 r 命令运行程序，并在断点处停下来。接下来用 s 命令单步执行：

^① 更严密的说法是：正整数集是满足(1)、(2)的最小集。这里牺牲一点严密性，换来的是更通俗易懂的表达方式。

```
(gdb) r
Starting program: C:\a.exe

Breakpoint 1, f (n=3) at factorial.c:3
3      return n == 0 ? 1 : f(n-1)*n;
(gdb) s

Breakpoint 1, f (n=2) at factorial.c:3
3      return n == 0 ? 1 : f(n-1)*n;
(gdb) s

Breakpoint 1, f (n=1) at factorial.c:3
3      return n == 0 ? 1 : f(n-1)*n;
(gdb) s

Breakpoint 1, f (n=0) at factorial.c:3
3      return n == 0 ? 1 : f(n-1)*n;
(gdb) s
4      }
```

看到了吗？在第一次断点处， $n=3$ （3 是 main 函数中的调用参数），接下来将调用 $f(3-1)$ ，即 $f(2)$ ，因此单步一次后显示 $n=2$ 。由于 $n=0$ 仍然不成立，继续递归调用，直到 $n=0$ 。这时不再递归调用了，执行一次 s 命令以后会到达函数的结束位置。

接下来该做什么？没错！好好看看下面的调用栈吧！

```
(gdb) bt
#0 f (n=0) at factorial.c:4
#1 0x00401308 in f (n=1) at factorial.c:3
#2 0x00401308 in f (n=2) at factorial.c:3
#3 0x00401308 in f (n=3) at factorial.c:3
#4 0x00401359 in main () at factorial.c:6
```

```
(gdb) s
```

```
4      }
```

```
(gdb) bt
```

```
#0 f (n=1) at factorial.c:4
#1 0x00401308 in f (n=2) at factorial.c:3
#2 0x00401308 in f (n=3) at factorial.c:3
#3 0x00401359 in main () at factorial.c:6
```

```
(gdb) s
```

```
4      }
```

```
(gdb) bt
```

```
#0 f (n=2) at factorial.c:4
#1 0x00401308 in f (n=3) at factorial.c:3
```

```
#2 0x00401359 in main() at factorial.c:6
```

```
(gdb) s
```

```
4      }
```

```
(gdb) bt
```

```
#0 f (n=3) at factorial.c:4
```

```
#1 0x00401359 in main() at factorial.c:6
```

```
(gdb) s
```

```
6
```

```
main() at factorial.c:7
```

```
7      return 0;
```

```
(gdb) bt
```

```
#0 main() at factorial.c:7
```

每次执行完 `s` 指令，都会有一层递归调用终止，直到返回 `main` 函数。事实上，如果在递归调用初期查看调用栈，则会发现每次递归调用都会多一个栈帧——和普通的函数调用并没有什么不同。确实如此。由于使用了调用栈，C 语言自然支持了递归。在 C 语言的函数中，调用自己和调用其他函数并没有任何本质区别，都是建立新栈帧，传递参数并修改当前代码行。在函数体执行完毕后删除栈帧，处理返回值并修改当前代码行。

提示 4-18：由于使用了调用栈，C 语言支持递归。在 C 语言中，调用自己和调用其他函数并没有本质不同。

如果仍然无法理解上面的调用栈，可以作如下的比喻。

皇帝（拥有 `main` 函数的栈帧）：大臣，你给我算一下 `f(3)`。

大臣（拥有 `f(3)` 的栈帧）：知府，你给我算一下 `f(2)`。

知府（拥有 `f(2)` 的栈帧）：县令，你给我算一下 `f(1)`。

县令（拥有 `f(1)` 的栈帧）：师爷，你给我算一下 `f(0)`。

师爷（拥有 `f(0)` 的栈帧）：回老爷，`f(0)=1`。

县令：（心算 `f(1)=f(0)*1=1`）回知府大人，`f(1)=1`。

知府：（心算 `f(2)=f(1)*2=2`）回大人，`f(2)=2`。

大臣：（心算 `f(3)=f(2)*3=6`）回皇上，`f(3)=6`。

皇帝满意了。

虽然比喻不甚恰当，但也可以说明一些问题。递归调用时新建了一个栈帧，并且跳转到了函数开头处执行，就好比皇帝找大臣、大臣找知府这样的过程。尽管同一时刻可以有多个栈帧（皇帝、大臣、知府同时处于“等待下级回话”的状态），但“当前代码行”只有一个。

读者如果理解了这个比喻，但仍不理解调用栈，不必强求，知道递归为什么能正常工作即可。设计递归程序的重点在于给下级安排工作。

4.3.4 段错误与栈溢出

至此，对 C 语言的介绍已近尾声。别忘了，我们还没有测试 `f` 函数。也许你会说：不

必了, 我知道乘法会溢出——算阶乘时, 乘法老是会溢出。可这次不一样了。把 `main` 函数的 `f(3)` 换成 `f(100000000)` 试试 (别数了, 有 8 个 0)。什么? 没有输出? 不对呀, 即使溢出, 也应该是个负数或者其他 “显然不对” 的值, 不应该没有输出啊!

`gdb` 再次帮了我们的忙。用 `-g` 编译后用 `gdb` 载入, 二话不说就用 `r` 执行。结果发现 `gdb` 报错了!

```
(gdb) r
Starting program: C:\a.exe

Program received signal SIGSEGV, Segmentation fault.
0x00401303 in f (n=99869708) at 4-6.c:3
3       return n == 0 ? 1 : f(n-1)*n;
```

`gdb` 中显示程序收到了 `SIGSEGV` 信号——段错误。这太让人沮丧了! 眼看本章就要结束了, 怎么又遇到一个段错误? 别急, 让我们慢慢分析。我保证, 这是本章最后的难点。

你有没有想过, 编译后产生的可执行文件里都保存着些什么内容? 答案是和操作系统相关。例如, `UNIX/Linux` 用的 `ELF` 格式, `DOS` 下用的是 `COFF` 格式, 而 `Windows` 用的是 `PE` 文件格式 (由 `COFF` 扩充而来)。这些格式不尽相同, 但都有一个共同的概念——段。

“段” (`segmentation`) 是指二进制文件内的区域, 所有某种特定类型信息被保存在里面。可以用 `size` 程序^① 得到可执行文件中各个段的大小。如刚才的 `factorial.c`, 编译出 `a.exe` 以后执行 `size` 的结果是:

```
D:\>size a.exe
      text    data     bss      dec     hex filename
   2756      740      224     3720     e88 a.exe
```

此结果表示 `a.exe` 由正文段、数据段和 `bss` 段组成, 总大小是 3720, 用十六进制表示为 `e88`。这些段是什么意思呢?

提示 4-19: 在可执行文件中, 正文段 (`Text Segment`) 用于储存指令, 数据段 (`Data Segment`) 用于储存已初始化的全局变量, `BSS` 段 (`BSS Segment`) 用于储存未赋值的全局变量所需的空

是不是少了点什么? 调用栈在哪里? 它并不储存在可执行文件中, 而是在运行时创建。调用栈所在的段称为堆栈段 (`Stack Segment`)。和其他段一样, 堆栈段也有自己的大小, 不能被越界访问, 否则就会出现段错误 (`Segmentation Fault`)。

这样, 前面的错误就不难理解了: 每次递归调用都需要往调用栈里增加一个栈帧, 久而久之就越界了。这种情况叫做栈溢出 (`Stack Overflow`)。

提示 4-20: 在运行时, 程序会动态创建一个堆栈段, 里面存放着调用栈, 因此保存着函数的调用关系和局部变量。

^① `Linux` 和 `Windows` 下的 `MinGW` 中都有这个程序。